SQL Intermediate - Cheat Sheet

All examples use this schema:

```
users        (id, name, email, created_at, status)
orders       (id, user_id, total, status, created_at)
order_items  (id, order_id, product_id, quantity, price)
products     (id, name, category, price)
```

All JOIN types in depth

A join combines rows from two tables based on a condition. The differences are about which rows survive when the match fails on one side.

Quick visual. Imagine two tables, `A` and `B`, with these rows:

A.id	B.id
1	1
2	2
3	
	4

INNER JOIN	LEFT JOIN	RIGHT JOIN	FULL OUTER
A.id B.id	A.id B.id	A.id B.id	A.id B.id
1 1	1 1	1 1	1 1
2 2	2 2	2 2	2 2
	3 NULL		3 NULL
		NULL 4	NULL 4

INNER JOIN

Only rows where the join key exists on both sides.

```sql
SELECT u.name, o.id AS order_id, o.total
FROM users u
INNER JOIN orders o ON o.user_id = u.id
WHERE o.status = 'paid';
```

LEFT (OUTER) JOIN

All rows from the left table. If there's no match on the right, those columns are NULL. Use this when you want "all users and their orders if any".

```sql
SELECT u.name, COUNT(o.id) AS order_count
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.name;
```

Users with zero orders still show up with `order_count = 0`.

RIGHT (OUTER) JOIN

Mirror of LEFT JOIN. You rarely see this in practice because most people just flip the table order and use LEFT JOIN.

```sql
SELECT u.name, o.id
FROM users u
RIGHT JOIN orders o ON o.user_id = u.id;
```

FULL OUTER JOIN

All rows from both sides. Where there's no match, the missing side is NULL. Useful for diffing two tables.

```sql
SELECT u.id AS user_id, o.id AS order_id
FROM users u
FULL OUTER JOIN orders o ON o.user_id = u.id;
```

MySQL doesn't support FULL OUTER directly. You emulate it with `LEFT JOIN UNION RIGHT JOIN`.

CROSS JOIN (Cartesian product)

Every row from A paired with every row from B. If A has 100 rows and B has 50, you get 5000 rows. Use it intentionally (generating combinations), or accidentally panic when your query returns a million rows.

```sql
SELECT u.id, p.id
FROM users u
CROSS JOIN products p;
```

SELF JOIN

The same table joined with itself. Classic case: employees with managers, where `manager_id` points back into the same table.

```sql
-- pretend users has a referrer_id column
SELECT u.name AS user_name, r.name AS referrer_name
FROM users u
LEFT JOIN users r ON r.id = u.referrer_id;
```

The aliases (`u` and `r`) are mandatory here, otherwise SQL can't tell which row you mean.

Anti-joins

"Rows in A with no match in B". Done with `LEFT JOIN ... WHERE other_side IS NULL`.

```sql
-- users who never placed an order
SELECT u.id, u.name
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL;
```

Semi-joins

"Rows in A that have at least one match in B", but you only want A's columns. Use EXISTS or IN. EXISTS is usually cleaner and safer with NULLs.

```sql
-- users who placed at least one paid order
SELECT u.id, u.name
FROM users u
WHERE EXISTS (
  SELECT 1 FROM orders o
  WHERE o.user_id = u.id AND o.status = 'paid'
);
```

Join order and the optimizer

You write the joins in some order, but the query planner is free to reorder them based on statistics (row counts, indexes, selectivity). For most queries, write what reads clearly and let the planner do its job. Optimizer hints exist (Oracle has them, Postgres has `pg_hint_plan`), but reach for them last.

Subqueries

Scalar subqueries

Returns one row, one column. Can sit anywhere a value can.

```sql
SELECT
  u.name,
  (SELECT MAX(o.total) FROM orders o WHERE o.user_id = u.id) AS biggest_order
FROM users u;
```

Subqueries in WHERE

```sql
-- users who bought anything in the 'books' category
SELECT id, name
FROM users
WHERE id IN (
  SELECT o.user_id
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.id
  JOIN products p ON p.id = oi.product_id
  WHERE p.category = 'books'
);
```

`NOT IN` has a gotcha. If the subquery returns even one NULL, the whole `NOT IN` returns nothing. Use `NOT EXISTS` instead:

```sql
-- users with no orders, NULL-safe version
SELECT id, name
FROM users u
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);
```

Subqueries in FROM (derived tables)

Treat a subquery's result like a table. Must be aliased.

```sql
SELECT category, AVG(order_value) AS avg_value
FROM (
  SELECT p.category, oi.price * oi.quantity AS order_value
  FROM order_items oi
  JOIN products p ON p.id = oi.product_id
) AS line_items
GROUP BY category;
```

Correlated subqueries

The inner query references a column from the outer query. Runs once per outer row, so watch performance on large tables.

```sql
-- each user's most recent order date
SELECT
  u.id,
  u.name,
  (SELECT MAX(o.created_at) FROM orders o WHERE o.user_id = u.id) AS last_order
FROM users u;
```

A window function or a JOIN with GROUP BY is usually faster.

EXISTS vs IN

Both can express semi-joins. Rules of thumb:

- EXISTS short-circuits as soon as it finds one match. Often faster on big tables.
- IN is fine for small, static lists.
- `NOT EXISTS` is the NULL-safe choice over `NOT IN`.

Common table expressions (CTEs)

A CTE is a named subquery that sits at the top of a statement. Makes complex queries readable.

```sql
WITH paid_orders AS (
  SELECT * FROM orders WHERE status = 'paid'
)
SELECT u.name, COUNT(po.id) AS paid_count
FROM users u
LEFT JOIN paid_orders po ON po.user_id = u.id
GROUP BY u.name;
```

Multiple CTEs chain together with commas:

```sql
WITH
paid_orders AS (
  SELECT * FROM orders WHERE status = 'paid'
),
big_spenders AS (
  SELECT user_id, SUM(total) AS lifetime_value
  FROM paid_orders
  GROUP BY user_id
  HAVING SUM(total) > 1000
)
SELECT u.name, bs.lifetime_value
FROM users u
JOIN big_spenders bs ON bs.user_id = u.id;
```

Recursive CTEs

For hierarchies (org charts, category trees) or generating sequences. 2 parts: the anchor (the base case) and the recursive step (references itself).

```sql
-- generate numbers 1 through 10
WITH RECURSIVE nums(n) AS (
  SELECT 1
  UNION ALL
  SELECT n + 1 FROM nums WHERE n < 10
)
SELECT n FROM nums;
```

Classic org-chart example. Pretend users has a `manager_id` column:

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, 1 AS depth
  FROM users
  WHERE id = 42  -- start from this user
  UNION ALL
  SELECT u.id, u.name, u.manager_id, c.depth + 1
  FROM users u
  JOIN chain c ON u.id = c.manager_id
)
SELECT * FROM chain;
```

CTE performance note

Older Postgres versions (pre-12) treated CTEs as an optimization fence. The planner materialized the CTE result before continuing. From Postgres 12 onward, simple CTEs get inlined like subqueries. You can force the old behavior with `WITH ... AS MATERIALIZED (...)`. MySQL 8+ inlines by default. Worth knowing if you inherit a query that performs differently across versions.

Window functions

Aggregate or ranking calculations that don't collapse rows. The output keeps one row per input row but adds a new column computed across a "window" of related rows.

Syntax:

```
function() OVER (
  PARTITION BY column
  ORDER BY column
  ROWS BETWEEN ... AND ...
)
```

Ranking functions

```
SELECT
  user_id,
  id AS order_id,
  total,
  ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY total DESC) AS rn,
  RANK()       OVER (PARTITION BY user_id ORDER BY total DESC) AS rnk,
  DENSE_RANK() OVER (PARTITION BY user_id ORDER BY total DESC) AS drnk,
  NTILE(4)     OVER (ORDER BY total) AS quartile
FROM orders;
```

Difference between them when there's a tie on `total = 100`:

total	ROW_NUMBER	RANK	DENSE_RANK
200	1	1	1
100	2	2	2
100	3	2	2
50	4	4	3

Offset functions

LAG looks at the previous row in the window. LEAD looks at the next row.

```sql
-- gap between consecutive orders per user
SELECT
  user_id,
  id,
  created_at,
  LAG(created_at) OVER (PARTITION BY user_id ORDER BY created_at) AS prev_order_at
FROM orders;
```

FIRST_VALUE and LAST_VALUE grab the first or last row in the window frame.

Aggregates as windows

Any aggregate (SUM, AVG, COUNT, MIN, MAX) works as a window function.

```sql
-- running total of revenue per day
SELECT
  DATE(created_at) AS day,
  SUM(total) AS day_revenue,
  SUM(SUM(total)) OVER (ORDER BY DATE(created_at)) AS running_total
FROM orders
WHERE status = 'paid'
GROUP BY DATE(created_at)
ORDER BY day;
```

Window frames

`ROWS BETWEEN` lets you control which rows in the partition are included.

```sql
-- 7-day moving average of order totals per user
SELECT
  user_id,
  created_at,
  total,
  AVG(total) OVER (
    PARTITION BY user_id
    ORDER BY created_at
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moving_avg_7
FROM orders;
```

Common frame clauses:

- `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (running total, the default with ORDER BY)
- `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` (last 7 rows)
- `ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING` (look ahead)

Set operations

Combine results of two SELECT statements that have the same column count and compatible types.

UNION vs UNION ALL

UNION removes duplicates. UNION ALL keeps them. UNION ALL is faster because it skips the dedup step. Use UNION ALL unless you actually need deduplication.

```sql
SELECT email FROM users WHERE status = 'active'
UNION ALL
SELECT email FROM newsletter_subscribers;
```

INTERSECT

Rows that appear in both result sets.

```sql
SELECT user_id FROM orders WHERE status = 'paid'
INTERSECT
SELECT user_id FROM orders WHERE status = 'refunded';
```

EXCEPT / MINUS

Rows in the first set that aren't in the second. Postgres calls it EXCEPT. Oracle calls it MINUS. MySQL has EXCEPT from 8.0.31 onward.

```sql
-- users who registered but never ordered
SELECT id FROM users
EXCEPT
SELECT user_id FROM orders;
```

NULL helpers

COALESCE

Returns the first non-NULL argument. Works with any number of arguments.

```sql
SELECT name, COALESCE(phone, email, 'no contact') AS contact
FROM users;
```

NULLIF

Returns NULL if the two arguments are equal, otherwise returns the first one. Handy for avoiding divide-by-zero:

```sql
SELECT total / NULLIF(quantity, 0) AS unit_price FROM order_items;
```

Dialect notes

- Postgres: COALESCE is standard. No IFNULL.
- MySQL: has both `IFNULL(a, b)` and COALESCE. IFNULL only takes two args.
- Oracle: `NVL(a, b)` for two args, `NVL2(a, b, c)` for if-then-else style.

Stick to COALESCE. It's portable and handles N arguments.

String functions

Concatenation

```sql
-- standard SQL and Postgres/Oracle
SELECT first_name || ' ' || last_name AS full_name FROM users;

-- portable across most databases
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM users;
```

In Postgres, `||` returns NULL if any operand is NULL. CONCAT treats NULLs as empty strings. Subtle but trips people up.

Common string operations

```sql
SELECT
  LOWER(email)                  AS lower_email,
  UPPER(name)                   AS upper_name,
  LENGTH(name)                  AS name_len,
  SUBSTRING(email FROM 1 FOR 5) AS prefix,
  TRIM(name)                    AS trimmed,
  REPLACE(name, ' ', '_')       AS slug
FROM users;
```

LIKE and ILIKE

LIKE is case-sensitive. `%` matches any sequence, `_` matches one character.

```sql
SELECT * FROM users WHERE email LIKE '%@gmail.com';
```

Postgres adds ILIKE for case-insensitive matching:

```sql
SELECT * FROM users WHERE name ILIKE 'john%';
```

In MySQL, default collation is often case-insensitive already, so plain LIKE behaves like ILIKE. Check your collation if behavior surprises you.

Regex

Postgres operators:

- `~` regex match (case-sensitive)
- `~*` regex match (case-insensitive)
- `!~` and `!~*` for negation

```sql
SELECT * FROM users WHERE email ~* '^[a-z]+@vendasta\.com$';
```

Oracle and MySQL use `REGEXP_LIKE(column, pattern)` or REGEXP operator.

Date / time functions

Current time

```sql
SELECT
  NOW(),                -- timestamp with timezone (Postgres)
  CURRENT_TIMESTAMP,    -- standard
  CURRENT_DATE,         -- date only
  CURRENT_TIME;         -- time only
```

Date arithmetic

Postgres uses INTERVAL:

```
SELECT
  created_at,
  created_at + INTERVAL '7 days'  AS expires_at,
  created_at - INTERVAL '1 month' AS one_month_ago
FROM users;
```

MySQL syntax differs slightly: `DATE_ADD(created_at, INTERVAL 7 DAY)`.

Extracting parts

```
SELECT
  EXTRACT(YEAR FROM created_at)  AS year,
  EXTRACT(MONTH FROM created_at) AS month,
  EXTRACT(DOW FROM created_at)   AS day_of_week,
  DATE_TRUNC('day',   created_at) AS day,    -- Postgres
  DATE_TRUNC('month', created_at) AS month_start  -- Postgres
FROM orders;
```

DATE_TRUNC is great for grouping by day, week, month. In MySQL, use `DATE_FORMAT(ts, '%Y-%m-%d')` or `DATE(ts)`.

Formatting

```
-- Postgres / Oracle
SELECT TO_CHAR(created_at, 'YYYY-MM-DD HH24:MI') FROM orders;

-- MySQL
SELECT DATE_FORMAT(created_at, '%Y-%m-%d %H:%i') FROM orders;
```

Constraints

Constraints enforce data rules at the database level. They're your last line of defense when application code has a bug.

PRIMARY KEY

Uniquely identifies a row. NOT NULL plus UNIQUE in one.

```
CREATE TABLE users (
  id    BIGSERIAL PRIMARY KEY,
  email TEXT NOT NULL
);
```

FOREIGN KEY

Enforces referential integrity. The referenced row must exist.

```
CREATE TABLE orders (
  id      BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  total   NUMERIC(10, 2),
  CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
);
```

Referential actions on delete or update:

- `CASCADE` : delete or update the child rows too.
- `SET NULL` : set the child's FK column to NULL (column must be nullable).
- `SET DEFAULT` : set it to the default value.
- `RESTRICT` / `NO ACTION` : block the delete if children exist.

UNIQUE

```
-- single column
ALTER TABLE users ADD CONSTRAINT uq_users_email UNIQUE (email);

-- composite: one user can have many addresses but each label only once
ALTER TABLE addresses
  ADD CONSTRAINT uq_addresses_user_label UNIQUE (user_id, label);
```

NULLs in UNIQUE columns are usually allowed multiple times (each NULL is distinct). Postgres 15+ adds `UNIQUE NULLS NOT DISTINCT` if you want NULLs treated as equal.

CHECK

Arbitrary boolean expression that every row must satisfy.

```
ALTER TABLE orders
  ADD CONSTRAINT chk_total_positive CHECK (total >= 0);

ALTER TABLE users
  ADD CONSTRAINT chk_status
    CHECK (status IN ('active', 'suspended', 'deleted'));
```

NOT NULL and DEFAULT

```
CREATE TABLE products (
  id         BIGSERIAL PRIMARY KEY,
  name       TEXT NOT NULL,
  price      NUMERIC(10, 2) NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Naming constraints

Always name your constraints. Anonymous constraints give error messages like `violates check constraint "products_total_check1"` which tell you nothing. Naming convention: `chk_`, `fk_`, `uq_`, `pk_` prefix plus table plus column. Saves hours during debugging.

Indexes (basics)

An index is a separate data structure (usually a B-tree) that maps column values to row locations. With an index, a lookup goes from O(n) full table scan to O(log n) tree walk.

Creating and dropping

```
CREATE INDEX idx_orders_user_id    ON orders(user_id);
CREATE INDEX idx_orders_created_at ON orders(created_at);
DROP   INDEX idx_orders_user_id;
```

When to add one

Add an index on columns that:

- Appear in WHERE clauses on big tables.
- Are used as join keys.
- You frequently ORDER BY.

A FK column is almost always worth indexing. Postgres doesn't auto-index FKs (unlike some other databases).

The tradeoff

Indexes speed up reads. They slow down writes (every INSERT, UPDATE, DELETE has to update the index too). They also take disk space. Don't add indexes "just in case". Add them when a query is slow and you can show the index helps.

What's coming in sql-advanced

The advanced cheat sheet covers EXPLAIN, B-tree internals, composite indexes and column order, covering indexes, partial indexes, expression indexes, and how the planner decides whether to use an

index. For now, “add an index on the column you filter or join by” gets you 80% of the way.

Views

A view is a saved query you can SELECT from like a table. It doesn’t store data, it just runs the underlying query each time you query the view.

```
CREATE VIEW active_users AS
  SELECT id, name, email
  FROM users
  WHERE status = 'active';

SELECT * FROM active_users WHERE email LIKE '%@vendasta.com';
```

Updatable views

If the view is simple enough (single table, no aggregates, no DISTINCT, no JOINs), you can INSERT/UPDATE/DELETE through it. Once you add aggregates or joins, the view becomes read-only. You can still make it writable with INSTEAD OF triggers, but at that point a stored procedure is usually clearer.

Dropping

```
DROP VIEW active_users;
```

Materialized views

A materialized view stores the result on disk. You refresh it manually with `REFRESH MATERIALIZED VIEW`. Great for expensive aggregations that don’t need to be real-time. Covered in `sql-advanced.pdf`.

Best practices

- Pick the simplest join that gives the right answer. Anti-joins via `LEFT JOIN ... IS NULL` are clear and fast.
- Prefer EXISTS / NOT EXISTS over IN / NOT IN when subqueries can contain NULL.
- Use UNION ALL unless you actually need deduplication.
- Name every constraint (`pk_`, `fk_`, `uq_`, `chk_`). Future-you reading error messages will thank you.
- Always alias derived tables and self-joined tables.
- Use CTEs for readability when a query has more than 2 layers of nesting.
- For “top N per group” problems, reach for window functions (ROW_NUMBER partitioned by group).

- Format SQL across multiple lines. One clause per line, indent join conditions.
- Use lowercase for SQL keywords or UPPERCASE consistently. Pick one per project.
- Index FK columns. Postgres doesn't auto-create those indexes.
- Use COALESCE for portable NULL handling instead of dialect-specific IFNULL / NVL.
- Use DATE_TRUNC (Postgres) for clean date-based grouping instead of string formatting.
- Wrap zero divisors with NULLIF to avoid runtime errors.

Common gotchas

- `NOT IN (subquery)` returns no rows if the subquery contains a single NULL. Use `NOT EXISTS` instead.
- `COUNT(column)` skips NULLs. `COUNT(*)` counts every row. They give different answers when the column is nullable.
- Aggregates in WHERE don't work. Use HAVING. WHERE filters rows before grouping, HAVING filters groups after.
- LEFT JOIN followed by a WHERE on the right table silently converts it back to an INNER JOIN. Put the filter in the ON clause if you want to keep the unmatched rows.
- UNION is slower than UNION ALL because of the dedup step. Default to UNION ALL.
- CROSS JOIN by accident (forgetting the ON clause in older syntax) explodes row counts.
- Window functions can't appear in WHERE. Wrap the query in a subquery or CTE and filter on the outside.
- LAST_VALUE with the default frame returns the current row, not the actual last row in the partition. You need `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.
- `||` returns NULL if any operand is NULL in Postgres. CONCAT treats NULL as empty string.
- Each NULL is distinct in a UNIQUE constraint by default. Two rows with NULL in the unique column both pass.
- DELETE without a WHERE clause wipes the whole table. Same with UPDATE. Always test with SELECT first.
- Self-joins need aliases. Forgetting them throws ambiguous column errors.
- MySQL doesn't support FULL OUTER JOIN directly. Emulate with `LEFT JOIN UNION RIGHT JOIN`.
- Adding an index on a large production table can lock writes. Use `CREATE INDEX CONCURRENTLY` in Postgres.

Quick reference

Feature	Syntax / Example	Notes
INNER JOIN	`A JOIN B ON A.id = B.a_id`	Only matching rows
LEFT JOIN	`A LEFT JOIN B ON ...`	All A rows, NULL for unmatched B
FULL OUTER JOIN	`A FULL OUTER JOIN B ON ...`	All rows from both. Not in MySQL
CROSS JOIN	`A CROSS JOIN B`	Cartesian product
Anti-join	`LEFT JOIN B WHERE B.id IS NULL`	A rows with no B match
Semi-join	`WHERE EXISTS (SELECT 1 FROM B WHERE ...)`	Use EXISTS over IN for NULL safety
Scalar subquery	`SELECT (SELECT MAX(x) FROM t) AS m`	Returns one row, one column
Correlated subquery	`... WHERE x = (SELECT ... WHERE inner.id = outer.id)`	Runs per outer row
CTE	`WITH t AS (SELECT ...) SELECT * FROM t`	Readable, reusable
Recursive CTE	`WITH RECURSIVE t AS (anchor UNION ALL recursive)`	Hierarchies, sequences
ROW_NUMBER	`ROW_NUMBER() OVER (PARTITION BY x ORDER BY y)`	Unique sequential
RANK / DENSE_RANK	`RANK() OVER (...)`	RANK skips after ties, DENSE_RANK doesn't
LAG / LEAD	`LAG(col) OVER (ORDER BY ...)`	Previous / next row in window
Running total	`SUM(x) OVER (ORDER BY d)`	Accumulates across order

Feature	Syntax / Example	Notes
UNION ALL	`q1 UNION ALL q2`	Faster, keeps duplicates
INTERSECT	`q1 INTERSECT q2`	Rows in both
EXCEPT / MINUS	`q1 EXCEPT q2`	Rows in q1 not in q2
COALESCE	`COALESCE(a, b, c)`	First non-NULL
NULLIF	`NULLIF(a, b)`	NULL if equal, else a
LIKE / ILIKE	`col LIKE 'abc%'`	ILIKE is case-insensitive (Postgres)
Regex (Postgres)	`col ~* 'pattern'`	`~*` case-insensitive
INTERVAL	`created_at + INTERVAL '7 days'`	Postgres syntax
DATE_TRUNC	`DATE_TRUNC('day', ts)`	Group by time bucket (Postgres)
FOREIGN KEY	`FOREIGN KEY (x) REFERENCES t(id) ON DELETE CASCADE`	Cascade, SET NULL, RESTRICT
CHECK	`CHECK (total >= 0)`	Arbitrary boolean condition
CREATE INDEX	`CREATE INDEX idx_x ON t(col)`	Use CONCURRENTLY in Postgres on big tables
CREATE VIEW	`CREATE VIEW v AS SELECT ...`	Saved query, not stored data